

XS WALLET

Technical Specification Document v2

Go Core + Atomic Swaps Desktop Application

Document	Value
Version	0.2.0
Status	Phase 1 Complete - Boltz Client Production-Ready
Date	January 20, 2026
Classification	Technical Specification
Target Audience	Developers, Architects, Auditors, CTO
Backend	Go Core (xscore) with gRPC
Swap Provider	boltz-backend (self-hosted, our liquidity)

Table of Contents

1. Executive Summary
2. System Architecture (Updated)
 - 2.1 Go Core + gRPC
 - 2.2 boltz-backend Self-Hosted
 - 2.3 2 RPC Primary Interfaces
3. Boltz Client Implementation
 - 3.1 HTTP Client (Production-Ready)
 - 3.2 WebSocket Client (Production-Ready)
 - 3.3 Status Normalization
4. Database Specification
5. Atomic Swap Protocol
6. Cryptographic Specifications
7. Security Model
8. Implementation Status
9. Roadmap

1. Executive Summary

XS Wallet is a self-custody desktop application enabling atomic swaps between Bitcoin (BTC), Liquid (L-BTC), and Lightning Network (LN) using Taproot and **boltz-backend (self-hosted)** as swap orchestrator with **our liquidity**. The system maintains true self-custody while providing seamless cross-chain and cross-layer swaps.

***Key Architecture Change (CTO Feedback):** The system now uses boltz-backend self-hosted instead of Boltz public API, with 2 primary RPC interfaces (LND gRPC + elementsd JSON-RPC), and bitcoind as runtime dependency only.*

1.1 Core Capabilities

Capability	Implementation	Status
Go Core (xscore)	Authoritative backend in Go with gRPC	Implemented
Boltz HTTP Client	Retry/backoff, Retry-After, typed errors	Production-Ready
Boltz WebSocket	Single-writer, gap-recovery, connCtx	Production-Ready
Status Normalization	Table-driven per swap type	Implemented
Swap Engine	CAS/optimistic locking, idempotency	Base Implemented
HD Wallet	BIP39/32/84/85 derivation	Partial
Encryption	Argon2id + AES-256-GCM	Designed

Table 1.1: Core Capabilities Matrix

1.2 Design Principles

Self-Custody: User controls all private keys; no server holds funds or signing authority.

Zero Trust: All external data (boltz-backend responses, node data) is verified locally.

Crash Recovery: All swap states are persisted; swaps can resume after any failure.

Authoritative State Machine: Go Core is the single source of truth for swap state (CTO endorsed as "coisa de gente cuidadosa").

2. System Architecture (Updated)

2.1 Go Core + gRPC

The backend has been refactored from Node.js to **Go Core (xscore)**, providing superior performance for cryptographic operations and native cross-platform support. Communication with frontend uses **gRPC** instead of Electron IPC, enabling future mobile clients.

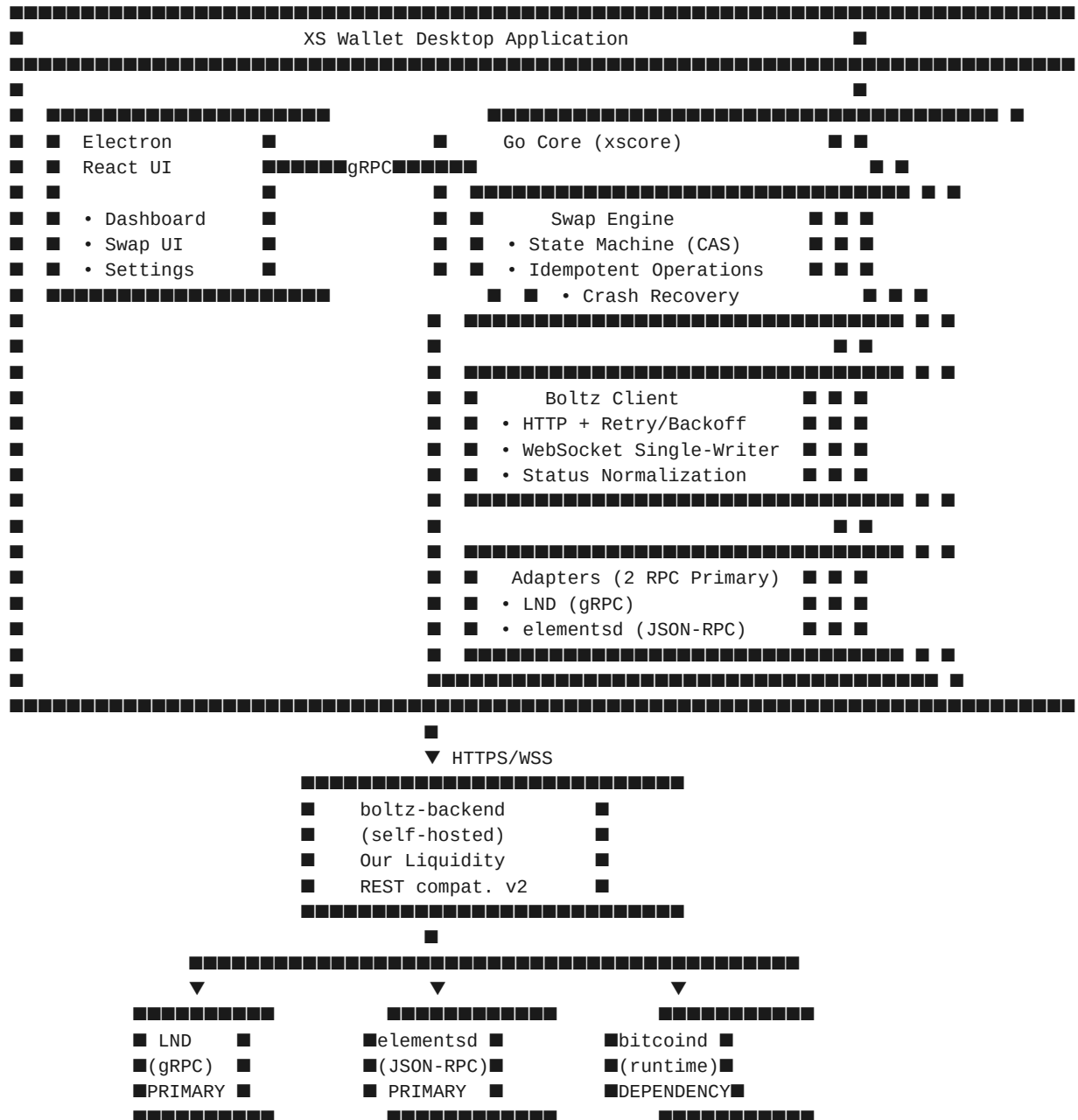


Figure 2.1: Updated System Architecture

2.2 boltz-backend Self-Hosted

Key Change: Instead of using Boltz public API as liquidity provider, the system uses **boltz-backend self-hosted** with our own liquidity. This provides:

Benefit	Description
---------	-------------

Internal Control	Full control over liquidity and swap orchestration
Reduced Dependencies	No external API availability concerns
API Compatibility	REST interface compatible with Boltz v2 spec
WebSocket Support	Real-time swap status updates

Table 2.1: boltz-backend Benefits

2.3 Two Primary RPC Interfaces (CTO Directive)

Per CTO feedback, the integration model has been simplified to 2 primary RPC interfaces:

Interface	Protocol	Purpose	Status
LND	gRPC	Lightning (invoices, payments, channels)	Primary
elementsd	JSON-RPC	Liquid (addresses, txs, blinding)	Primary
bitcoind	-	Runtime dependency (not direct RPC)	Dependency

Table 2.2: RPC Interfaces

Rationale: "o lnd em si já faz manejo on-chain e lightning" (CTO) - LND already handles on-chain operations, eliminating the need for direct bitcoind RPC from the wallet.

3. Boltz Client Implementation

The Boltz Client has been implemented with production-grade reliability standards. All components are tested and ready for integration with boltz-backend.

3.1 HTTP Client (Production-Ready)

Feature	Implementation	File
Exponential Backoff	$\text{initialBackoff} * 2^{\text{attempt}}$ (max 10s)	client.go
Retry-After Header	Parses and respects 429 header	client.go
Resource Leak Prevention	Body closed in closure, not defer in loop	client.go
Typed Errors	ErrSwapNotFound, ErrPairHashMismatch, etc	errors.go
All v2 Endpoints	/v2/swap/submarine, /reverse, /chain, etc	client.go

Table 3.1: HTTP Client Features

```
// Body closed in closure - prevents leak in retry loop
respBody, statusCode, err := func() ([]byte, int, error) {
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, 0, err }
    defer resp.Body.Close() // Closed HERE, not in outer loop!
    return io.ReadAll(resp.Body)
}()
```

Listing 3.1: Resource Leak Prevention Pattern

3.2 WebSocket Client (Production-Ready)

Feature	Implementation	Rationale
Single-Writer	writeMu sync.Mutex	Prevents concurrent write races
connCtx per Connection	Context canceled on disconnect	Prevents goroutine duplication
Gap Recovery	REST poll on reconnect	Catches missed updates
Channel swap.update	Correct v2 WebSocket format	Per Boltz spec
Ping/Pong Heartbeat	30s interval	Connection keepalive

Table 3.2: WebSocket Client Features

3.3 Status Normalization

Status normalization is table-driven per swap type, mapping Boltz status strings to internal state machine states with appropriate actions.

Boltz Status	Swap Type	Internal State	Action
transaction.claim.pending	Submarine	StateSigningMusig2Partial	Sign

invoice.settled	Reverse	StateCompleted	None
transaction.mempool	Reverse	StateSigningMusig2Partial	Sign
invoice.failedToPay	Submarine	StateFallbackScriptReady	Refund
swap.expired	All	StateFailed	None

Table 3.3: Status Normalization Examples

Critical Fix: *transaction.claim.pending* (not *invoice.paid*) is the correct trigger for MuSig2 partial signature in submarine swaps.

4. Database Specification

4.1 Schema Design

The database uses SQLite with Write-Ahead Logging (WAL) for optimal read/write concurrency. All swap state mutations use Compare-And-Swap (CAS) with version-based optimistic locking.

Table	Primary Key	Purpose
swaps	id (TEXT)	Authoritative swap state with CAS version
swap_events	seq (AUTOINCREMENT)	Immutable audit log for debugging
swap_ops	(swap_id, op_key)	Idempotent operation ledger
utxo_reservations	(chain, txid, vout)	Prevents UTXO double-spend
ln_reservations	payment_hash_hex	Prevents duplicate LN payments
app_config	key	User configuration (snapshot in locked_intent)

Table 4.1: Database Tables

4.2 CTO-Endorsed Design Patterns

The following patterns were specifically praised by CTO as "coisa de gente cuidadosa":

Pattern	Implementation	Benefit
locked_intent	Immutable JSON snapshot of quote/fees	Audit trail; prevents mutation
CAS with version	UPDATE ... WHERE version = ?	Prevents race conditions
swap_ops ledger	INSERT OR IGNORE + SELECT	Safe retries; no duplicate actions
utxo_reservations	Reserve before broadcast	No double-spend across swaps
ln_reservations	Reserve before pay	No duplicate LN payments

Table 4.2: Production-Ready Patterns

5. Atomic Swap Protocol

5.1 Responsibility Split: Wallet vs boltz-backend

The responsibility split between wallet and boltz-backend is being documented in the current spike. Key points:

Operation	Wallet	boltz-backend
Preimage generation (reverse/chain)	✓	
HTLC script verification	✓	
Funding HTLCs	✓	
Claim HTLCs (reveal preimage)	✓ (reverse/chain)	✓ (submarine)
LND payment	✓	✓
Liquidity management		✓
WebSocket status updates	Consumer	Producer

Table 5.1: Responsibility Split

5.2 State Machine

State	Description	Transitions
open	Quote accepted, not locked	locked canceled
locked	Parameters locked, intent immutable	commit_started canceled
commit_started	Funding broadcast	waiting failed
waiting	Awaiting counterparty	waiting_claim_details refund_coop_waiting
signing_musig2_partial	Creating partial sig	sent_partial_to_provider
waiting_provider_broadcast	Provider broadcasting	completed fallback_script_ready
fallback_script_ready	Script-path fallback	refunding completed
completed	Terminal success	(terminal)
failed	Terminal failure	(terminal)

Table 5.2: Swap State Machine

6. Cryptographic Specifications

6.1 Vault Encryption

Parameter	Value	Rationale
KDF	Argon2id	Memory-hard; GPU/ASIC resistant
Memory	64 MB	Desktop-appropriate security/performance
Iterations	3	Standard recommendation
Cipher	AES-256-GCM	Authenticated encryption
IV Length	12 bytes	GCM standard

Table 6.1: Vault Encryption Parameters

6.2 MuSig2 + Taproot

Component	Specification
Key Order	Provider pubkey first, then user (deterministic)
Nonce Generation	Deterministic from session + private key
Session Persistence	musig_* fields in swaps table
Key-Path Spend	MuSig2 aggregated signature (cooperative)
Script-Path Fallback	HTLC conditions if cooperation fails

Table 6.2: MuSig2 Specification

7. Security Model

Threat	In Scope	Mitigation
Seed theft (offline)	Yes	Argon2id + device secret (v1.1)
PIN brute force	Yes	Exponential backoff; lockout
Swap manipulation	Yes	Local script verification
Race conditions	Yes	CAS + idempotency ledger
Double broadcast	Yes	swap_ops table
Duplicate LN payment	Yes	In_reservations table

Table 7.1: Threat Model

8. Implementation Status

Current progress: **~40% of MVP complete**. Phase 1 (Boltz Client) is production-ready.

Component	Files	Status
Go Core (xscore)	cmd/xscore/main.go	Implemented
Boltz HTTP Client	internal/boltz/client.go	Production-Ready
Boltz WebSocket	internal/boltz/ws.go	Production-Ready
Status Normalization	internal/boltz/status.go	Production-Ready
Types (MinerFeesAny)	internal/boltz/types.go	Production-Ready
Errors	internal/boltz/errors.go	Production-Ready
Provider Interface	internal/boltz/boltz.go	Production-Ready
Swap Engine	internal/swap/engine.go	Base Implemented
Database Layer	internal/db/db.go	Implemented
LND Adapter	internal/adapters/lnd/	Next Sprint
Liquid Adapter	internal/adapters/liquid/	Next Sprint

Table 8.1: Implementation Status

8.1 Test Results

All Boltz Client tests passing:

```
$ go test -v ./internal/boltz/...

=== RUN    TestMinerFeesAny_UnmarshalNumber
--- PASS: TestMinerFeesAny_UnmarshalNumber (0.00s)
=== RUN    TestMinerFeesAny_UnmarshalObject
--- PASS: TestMinerFeesAny_UnmarshalObject (0.00s)
=== RUN    TestNormalizeSubmarine_ClaimPending
--- PASS: TestNormalizeSubmarine_ClaimPending (0.00s)
=== RUN    TestAllSubmarineStatuses
--- PASS: TestAllSubmarineStatuses (0.00s)

ok  github.com/xs-wallet/xscore/internal/boltz 0.563s
```

Listing 8.1: Test Results

9. Roadmap

Phase	Duration	Deliverables	Status
Boltz Client	Complete	HTTP, WebSocket, Status Normalization	✓ Done
Spike boltz-backend	Current	Responsibility Split, Integration Test	In Progress
Adapters	1 week	LND gRPC, elementsd JSON-RPC	Next
Wiring	1 week	xscore → boltz-backend, E2E tests	Pending
Frontend	2 weeks	Migrate BRLN devdash, Electron integration	Pending

Table 9.1: Implementation Roadmap

9.1 Post-MVP Roadmap

Version	Target	Features
v1.1	Q2 2026	Hardware wallet, SQLCipher, device secret
v1.2	Q3 2026	Coinjoin, Payjoin, LNURL
v2.0	Q4 2026	EVM Module (MetaMask + LI.FI) - isolated

Table 9.2: Post-MVP Roadmap

Note on EVM Module: Per CTO direction, MetaMask + LI.FI integration will be implemented as an isolated module, separate from the UTXO/LN core, to avoid contaminating the atomic swap architecture.